LAB MANUAL

From NLP to LLM

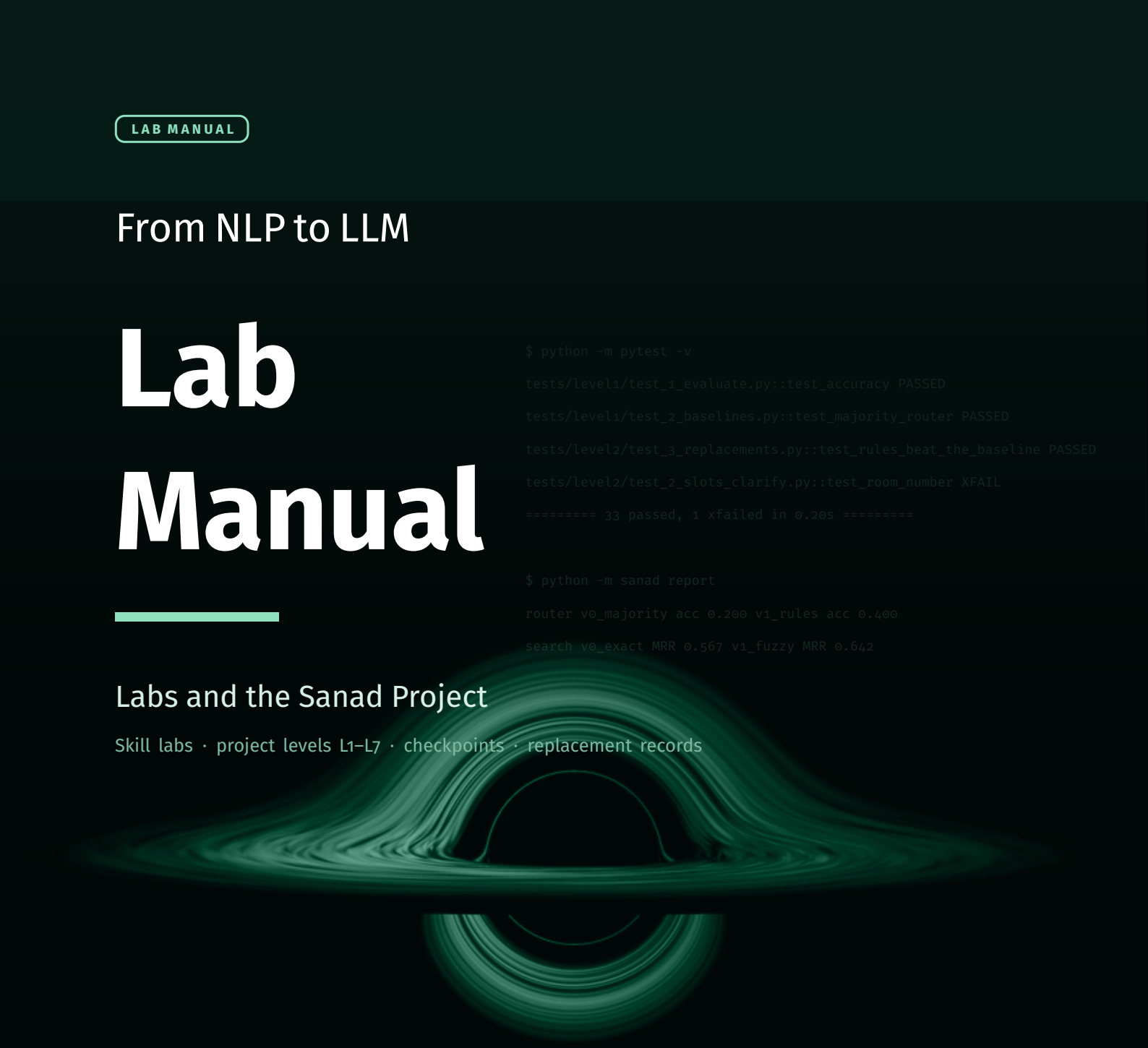
Lab Manual

```
$ python -m pytest -v
tests/level1/test_1_evaluate.py::test_accuracy PASSED
tests/level1/test_2_baselines.py::test_majority_router PASSED
tests/level2/test_3_replacements.py::test_rules_beat_the_baseline PASSED
tests/level2/test_2_slots_clarify.py::test_room_number XFAIL
========= 33 passed, 1 xfailed in 0.20s =========

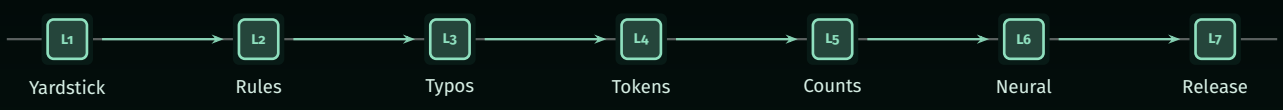
$ python -m sanad report
router v0_majority acc 0.200 v1_rules acc 0.400
search v0_exact MRR 0.567 v1_fuzzy MRR 0.642
```

Labs and the Sanad Project

Skill labs · project levels L1–L7 · checkpoints · replacement records



ONE SYSTEM · BUILT A LEVEL AT A TIME



Mahmmoud A. Mahdi

FIRST EDITION · 2026



THIS LAB MANUAL BELONGS TO

Student name _____

Student ID _____

Group / section _____

Contents

How to use this manual v

I Skill labs 1

1 An Arabic normaliser built from patterns 3

2 An Arabic ELIZA 5

3 Corpus statistics and an Arabic bigram model 7

II The Sanad project 11

L1 The yardstick 13

L1.1 What you build 13
L1.2 Steps 14
L1.3 Your record 14

L2 Rules, slots, and a clarifier 15

L2.1 What you build 15
L2.2 Steps, in four checkpoints 16
L2.3 Your record 17

L3 Corpus counts and a typo-tolerant search 19

L3.1 What you build 20
L3.2 Steps, in four checkpoints 20
L3.3 Your record 20

L4 Units of text, and a first keyboard 23

L4.1 What you build 23
L4.2 Steps 24
L4.3 Your record 25

A Project data and configuration 27

How to use this manual

This manual holds the coding work of *From NLP to LLM*. Read a chapter of the book first, then come here and build what it taught. There are two kinds of work, and they do different jobs.

Skill labs are short drills. Lab 1 and Lab 2 follow Chapter 2 of the book; Lab 3 has a part after Chapter 4 and a part after Chapter 5; Lab 4 follows Chapter 7, with later parts after Chapters 8 and 9. Labs 3 and 4 separate core tasks from challenge tasks, and each route has its own test command. Each is a separate starter project with its own tests, so you can do a lab without the others.

The Sanad project is one system that grows at the end of every chapter. Sanad is the university help-desk assistant you have met in the book's comics, and Nour is still the student whose messages break it. Sanad has three services that share one way of reading text:

- the **router** turns a ticket into an intent, a course code, an amount, and a reply or a question;
- the **search** returns the three FAQ passages most likely to answer a question;
- the **keyboard** suggests the next word while Nour types.

Every service reads its text through the same two shared parts, a normaliser and then a tokenizer. Replacing a shared part therefore changes the scores of every service at once, and noticing that is part of the work.

What to do after each chapter

The manual groups the labs together and the project levels together, so that each is easy to find again. Work through them in this order. Worksheets are in the student workbook; a project log is the workbook page that records a project level.

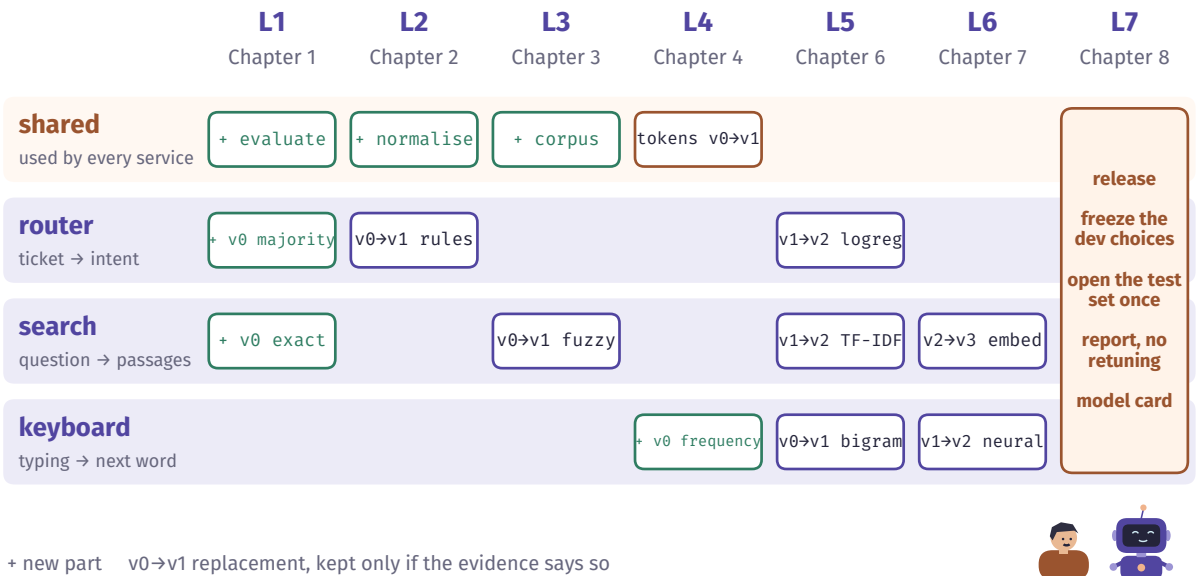


Figure 1 The Sanad project. Each column is the level at the end of one book chapter; Chapter 5 adds none. A green chip adds a part; an arrow chip replaces an older version of a part, but only if the evidence on the dev split supports it. Level 7 replaces nothing: it freezes the choices made on dev, opens the test split once, and reports the result without retuning.

After	Workbook	Skill lab	Sanad project
Ch. 1	Sheets 1–2, log L1	—	Level 1
Ch. 2	Sheets 3–4, log L2	Lab 1, then Lab 2	Level 2
Ch. 3	Sheet 5, log L3	Lab 3 handed out: its corpus (D: CHALLENGE)	Level 3
Ch. 4	Sheets 6–8, log L4	Lab 3, part C	Level 4
Ch. 5	Sheet 9	Lab 3, G1–G6 (G7: CHALLENGE)	No new level
Ch. 6	Sheets 10–11	—	Level 5
Ch. 7	Sheets 12–15	Lab 4, P1–P2, R1 (R2–R3, S: CHALLENGE)	Level 6
Ch. 8	Sheets 16–20	Lab 4, U1 (U2–U3: CHALLENGE)	Level 7: release
Ch. 9	Sheet 21	Lab 4, part T	Evaluate your latest level

Replace, measure, then decide

A new chapter usually teaches a better way to do something Sanad already does. The project never overwrites the old way. Each version of a part is its own file, such as `router/v0_majority.py` and `router/v1_rules.py`, and one line of `sanad.toml` chooses which runs. Before you change that line, run both versions on the same dev split and write an entry in `REPLACEMENTS.md`: the scores, one failure the new version

fixes, one it introduces, and your decision. Keeping the old version is a valid result when the evidence says so.

Checkpoints: you can always continue

Each level starts from a checkpoint, `sanad-Lk.zip`. It contains working reference code for every earlier level and empty stubs for level k, so it is released only after level $k-1$ is due. Carry your own code forward if you prefer; take the checkpoint if you fell behind or want a clean start. Either way, the tests of the earlier levels pass, and the tests of level k fail until you finish it. You need Python 3.11 or newer. Each `sanad-Lk.zip` extracts to a folder named `Lk`: `sanad-L1.zip` gives `L1`.

```
cd L1                                                    # L2, L3, ... for
    later levels
python3 -m venv .venv && source .venv/bin/activate   # Windows: .venv\
    Scripts\activate
pip install -r requirements-dev.txt                   # pytest, and sanad
    itself
python -m pytest -v                                    # earlier levels
    pass
python -m sanad report                                 # every version, on
    dev
```

Rules of the data

The tickets are constructed for teaching: 115 help-desk messages in English, Modern Standard Arabic, Egyptian Arabic, and Arabizi, split into 75 training, 20 dev, and 20 test tickets (Appendix A). Write rules and choose settings from the training tickets, compare versions on dev, and leave the test split closed: the code refuses to load it until level 7.

Reading this manual

Numbers such as Section 1.4.2 or Figure 1.11 point into the book. The manual's own chapters are the labs, the project levels, and an appendix. Work marked CHALLENGE belongs to the challenge route: it is optional, has its own tests, and is never needed for a complete core submission. Instructor notes appear only in the instructor edition.

PART

I

Skill labs

LAB

1 An Arabic normaliser built from patterns



After book Chapter 2, Section 2.1

The starter project is `labs/lab01`. It follows the layout of every lab in this manual: code in `src/arabic_nlp/`, tests in `tests/`, and pytest settings in `pyproject.toml`. Every function in the starter has a docstring contract and raises `NotImplementedError`.

Two kinds of test. `tests/test_public_checks.py` is yours: it runs in full, and its expected values all come from the book. Use it to check each step. The other files in `tests/` are the *graded* tests; in your copy they show as *skipped*, and your instructor runs them when marking. A passing public check means "ready for the next step", not "full marks".

```
cd labs/lab01
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements-dev.txt
python -m pytest tests/test_public_checks.py -v
```

1. Record `python --version` and `python -m pytest --version`. Run the public checks once and note the number of failures before you write any code: all eight fail at first.
2. Compile the patterns at the top of `normalize.py` from the code points in Table 2.2. Then implement N1–N5. Predict each expected value, then check with `python -m pytest tests/test_public_checks.py -k "n1 or n3"`.
3. Implement the tokenizer N6 and the extractors N7. Explain why the order of the three alternatives in `TOKEN` matters for *3ayez*: what do you get if the number alternative comes before the Latin one? Then explain why `2024` comes out the same in either order. Check: `-k "n6 or n7"`.
4. Implement N8 `strip_clitics`, and check with `-k n8`. The three `known_failure` checks are marked `xfail(strict=True)`: when your rule follows the written policy they show as `xfailed`, which is the correct result. If one *passes*, your rule differs from the written policy. For each one, explain which property of Arabic the string rule cannot see. Reproducing a written policy and improving a task are separate experiments. To fix a failure, write a new policy version, such as N8b, give it its

own expected results, and compare both versions on the same held-out messages. Repairing a known failure is good work; doing it silently under the old policy name is what the strict tests prevent.

5. Add two tests of your own, in a new file such as `tests/test_mine.py`: one input the policy must accept and one it must reject. Leave the supplied test files unchanged.
6. Measure `find_course_codes` on the supplied set: run `python evaluate_codes.py`. It scores your function on `data/course_codes.tsv`, the 13 annotated messages of book Example 2.2, and lists every message where it is wrong. Because everyone uses the same set, your scores can be compared. Note the scoring contract: `find_course_codes` returns code values without positions, so the script compares values within each message. That is looser than the book's exact-span scoring (Figure 2.6): returning the right code from the wrong place in a message still counts as correct here. Then copy the file, add five messages of your own (include at least one look-alike, such as a room number), and run `python evaluate_codes.py` with your file's path. Explain why your scores differ from the book's, whose pattern reads ASCII digits only.

Submit: the final `pytest -v` output, your two added tests, the scores on the supplied set and on your extended set, and a short paragraph that names one task for which N3–N4 would be harmful.

LAB

2

An Arabic ELIZA



After book Chapter 2, Section 2.3

The starter project `labs/lab02` already contains the rule table `RULES`, the reflection dictionary, and the fallback replies. You implement the engine. As in Lab 1, `tests/test_public_checks.py` runs in full and is yours to check each step; the other test files are graded and show as skipped in your copy:

```
python -m pytest tests/test_public_checks.py -v
```

1. Implement `clean` (E1): remove marks and tatweel, collapse spaces, and trim final punctuation, including the Arabic question mark.
2. Implement `reflect_word` and `reflect` (E3), and check with `-k e3`. The nisba check is marked `xfail(strict=True)`: it should show as `xfailed`. Write one sentence on why it fails.
3. Implement `select_rule` (E2) and `respond` (E4), and check with `-k "e2 or e4"`. The reply template is chosen by `turn`, so a run is repeatable without random numbers.
4. Implement `converse`: it takes a list of user lines and returns one reply per line, numbering the turns from 0, so the first line is turn 0. It is a few lines long; check it with `-k converse`. Then run all the public checks: four pass and one is `xfailed`.
5. Add a rule of your own with a rank, at least two templates, and a test in a new file (such as `tests/test_mine.py`) showing it beats a lower-ranked rule. A good candidate is a rule for registration (تسجيل, *tasjil*).
6. Run `converse` on five messages of your own, including one Egyptian and one feminine self-description. Classify each reply as useful, harmless but empty, or wrong, and explain the mechanism behind each wrong reply.

Submit: the `pytest -v` output, your new rule and test, and the five-turn transcript with your classification.

LAB

3 Corpus statistics and an Arabic bigram model



Handed out with book Chapter 3 (its corpus; part D is a challenge); part C after Chapter 4; part G after Chapter 5; G7 is a challenge

The starter project `labs/lab03` contains the 40-request corpus from Chapter 3 in `data/corpus_ar.txt` and three modules to complete. Set it up as in Lab 1. The lab has a core route and a challenge route, and each has its own tests, so an unfinished challenge never spoils a core submission.

Route	Tasks	Tests to run
Core, after Ch. 4	Part C: C1–C4 (step 1)	`python -m pytest tests/test_1_corpus.py`
Core, after Ch. 5	Part G: G1–G6 (steps 2–6)	`python -m pytest tests/test_2_ngrams.py -k "not test_g7"`
CHALLENGE	G7, interpolation (steps 7–8)	`python -m pytest tests/test_2_ngrams.py -k test_g7`
CHALLENGE	Part D, edit distance, backtrace, WER (steps 9–10)	`python -m pytest tests/test_3_edit.py`

D1–D2 repeat the edit distance that project level 3 already requires, so you may port your Sanad `edit.py` and run its tests as a check; D3–D6 go beyond it.

Evaluation protocol. Fix the splits before step 5 and keep them for the whole lab. Requests 1–32 are for fitting; the last eight are the held-out set that step 5 scores. The challenge route later divides requests 1–32 into training (1–24) and development (25–32) to choose a weight. Step 5 has already looked at the last eight requests, so the challenge route's final score on them is a demonstration of the protocol, not fresh held-out evidence.

Core route.

1. **Part C: corpus statistics** (`corpus.py`, C1–C4), after Chapter 4. Implement `tokens`, `read_corpus`, `type_token`, `zipf_table`, `hapax`, and `heaps_curve`. The tests fix $N =$

185 and $V = 105$, and $V = 102$ after normalisation. Before running each test, predict whether normalisation can change N.

2. **N-gram windows** (`ngrams.py`, G1), after Chapter 5. Implement `ngrams` with this chapter's padding: $n-1$ start markers and one end marker for $n > 1$, with no padding for unigrams.
3. **Bigram model** (G2–G5). Implement `BigramLM.fit`, `map`, `prob`, `logprob`, `perplexity`, and `generate`. The tests use the three requests of Figure 5.11, so you can check every number by hand first. `prob` returns zero for any target outside the fitted support. With `unk=True`, `fit` also reserves `<unk>`, and `map` sends unseen words to it, following Section 5.3.2.
4. **Sampling** (G6). Implement `sample(rng, max_len)` with the inverse-CDF rule of Section 5.4.1: order the support by token text, draw u from `rng.random()`, and return the words with a status, `end` or `budget`. Before running the tests, predict the sample for `random.Random(3)` from the three requests by hand.
5. **Experiment.** Hold out the last eight requests and fit every model on the first 32. Fix the support before you look at the held-out words. First fit a closed-vocabulary add-one model and score the held-out set; explain why its perplexity is infinite even though it is smoothed. Then fit with `unk=True`, unsmoothed and add-one, with and without normalisation. For each of the four settings report V, the number of held-out targets, the number mapped to `<unk>`, and the perplexity. For one held-out request, name the first word outside the fitted vocabulary and the first transition that was never seen in training.
6. **Read the output.** Greedy generation from the full corpus produces the fluent request "I want to add a course to the next week". Show which two training requests it splices together, and connect this to the ELIZA effect. Then draw five samples with `random.Random(1)` and say which are copies of training requests and which are new combinations.

Challenge route CHALLENGE

7. **Interpolation** (G7) CHALLENGE. Implement `InterpolatedLM`, which mixes the bigram row with an add-one unigram over the same support, and `choose_lambda`, which picks a weight from a grid by development perplexity (Sections 5.3.3 and 5.3.4). A context never seen in training has no bigram row: say why the unigram alone keeps that row summing to one.
8. **Choose a weight on development data** CHALLENGE. Fit an unsmoothed bigram with `unk=True` on requests 1–24 and choose λ on requests 25–32 with `choose_lambda`. Score an add-one bigram with `unk=True` on the same development split. Decide which model to keep from the development scores alone, before you look at any held-out result. Then refit it on requests 1–32 and score the last eight requests once. Because step 5 already inspected those requests, label this final score as a

demonstration of the protocol, not as new evidence. With a new corpus, fix training, development, and test splits before the first experiment (Chapter 8). Report every development score, your decision, and the final score with that label.

9. **Part D: edit distance** (D1–D3) CHALLENGE. Implement `distance_table`, `levenshtein`, and `closest` in `edit.py`. Check your table first against the hand-filled *exma→exam* table of Example 3.3, then reproduce Figure 3.9 from the table your code returns. Then implement `weighted_distance` (D3) with per-symbol costs, following Section 3.2.3. Its border cells must accumulate the real deletion and insertion costs. Under the book's directed policy, مدرسة with taa marbuta becomes the haa spelling at cost 0.25, but the reverse change costs 1. Explain why that asymmetry means a weighted score need not be a distance in the mathematical sense.
10. **Part D: alignment and word error rate** (D4–D6) CHALLENGE. Implement `align` (D4), the backtrace of Section 3.2.4: it must return `IMMMI` for Figure 3.9 and break ties towards the diagonal. Build `wer` and `corpus_wer` (D5) on top of it. Before you run the tests, compute by hand the pooled and the averaged WER of the two pairs in Figure 3.16; your `corpus_wer` must return the pooled one. Then implement `swap_distance` (D6). Try the swap *while* the table is filled: a version that adds swaps to a finished Levenshtein table still passes for *exma*, where the swap is the last move, but fails for مدسرة→مدرسة. Explain why.

Submit, core route, in stages. After Chapter 4: the output of the part C tests, and a five-line data statement for `corpus_ar.txt` following Section 3.1.1 (purpose, source, varieties, what it may not be used to claim, and normalisation). After Chapter 5: the output of the G1–G6 tests; your perplexity table from step 5 with the split, support, and policies stated; your answers to step 6; and one paragraph on the effect of normalisation on sparsity in this corpus. Say whether the two normalisation settings score the same prediction problem; Chapter 8 returns to that question.

CHALLENGE **Submit, challenge route, in addition:** the output of the challenge test commands you attempted, the development-weight table, your keep-or-replace decision, and the final score, labelled as a demonstration.

PART

II

The Sanad project

L1

PROJECT LEVEL

The yardstick

IN THIS CHAPTER

L1.1 What you build 13
L1.2 Steps 14
L1.3 Your record 14

After book Chapter 1: Sections 1.4.2–1.4.4

Before Sanad can get better, you need a way to tell whether it did. This level builds the evaluation harness every later level is judged with, and the two weakest systems that can still be measured: a router that always gives the same answer, and a search that only matches exact words.

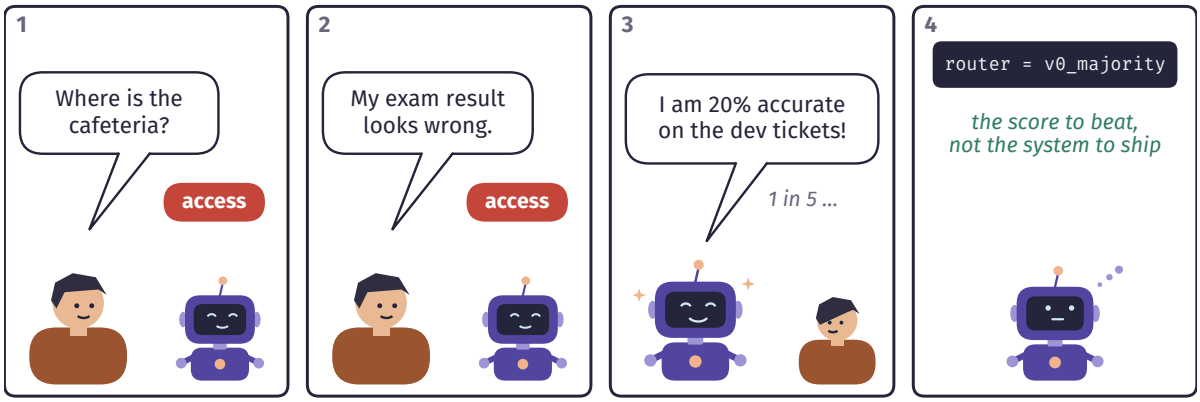


Figure L1.1 Sanad's first router gives every ticket the same label, *access*, and scores 20% on dev: the baseline to beat, not a system to ship. The numbers are the reference run.

Sanad routes tickets to five intents, not the four of book Chapter 1: it adds *fees* for payment and refund questions. This is the project's own contract, separate from the chapter's worked router, so expect five rows and columns in its confusion matrix.

L1.1 What you build

- `CONTRACT.md`: a task contract for each of the three services.
- `sanad/evaluate.py`: accuracy, the confusion matrix, per-class precision and recall, and the majority baseline. The same file supplies, ready-made, the measures the

reports also print: F_β, macro and micro averages, the rough 95% margin, precision@k, and reciprocal rank. Book Chapter 6 explains them (Sections 6.2.2 and 6.5); read the supplied code now and return to it then.

- `router/v0_majority.py`: always answer the most frequent training intent.
- `search/v0_exact.py`: rank passages by the number of question words they contain.

L1.2 Steps

1. Unzip `sanad-L1`, set it up as in *How to use this manual*, and run the tests once. Record how many fail before you write any code. Expect 8 failures and 3 passes: the failures are the functions you have not written yet, and the finished reference passes all 11.
2. Write `CONTRACT.md`. For each service state the input, the output, who produces the reference, the language varieties, the most harmful error, the metric, and the fallback. Adapt the help-desk specification of book Example 1.4. For the router, describe in words which intent-routing errors cost most and why, for example a fees ticket sent to *other*. Choosing a measure that weights those errors waits for book Chapter 6. For the keyboard, which you build at level 4, the contract is all you write now.
3. Implement the four unfinished functions in `evaluate.py`. Run the tests in `tests/level1/test_1_evaluate.py` until they pass. The tests use the ten tickets of book Figure 1.11, so predict each value from the book before running. Note how `None`, a router that declines to answer, enters the counts: it is a miss for the true class and a false alarm for none.
4. Implement `Router.fit` and `Router.predict` in `v0_majority.py`. All five intents occur 15 times in the training split. Predict the label your router will choose and its dev accuracy before you run `test_2_baselines.py`.
5. Implement `v0_exact.py`. Then run `python -m sanad report` and copy both tables into `REPLACEMENTS.md` as your level-1 baselines.
6. Read the search table by variety. Explain why Modern Standard Arabic questions find their passage and Arabizi questions almost never do, although the passages contain both English and Arabic.

L1.3 Your record

Level 1 replaces nothing. Its record is the baseline every later level must beat on the same 20 dev tickets: accuracy with its margin, macro-F_1, P@3, and MRR, each by variety. Accuracy is yours; the other measures come from the supplied functions until Chapter 5 explains them.

Submit: the `pytest -v` output, `CONTRACT.md`, the `report` tables, and two sentences on the Arabizi search gap.

PROJECT LEVEL

L2

Rules, slots, and a clarifier

IN THIS CHAPTER

L2.1 What you build 15
L2.2 Steps, in four checkpoints 16
L2.3 Your record 17

After book Chapter 2: Sections 2.1 and 2.3

Chapter 2 gave you patterns and ELIZA. This level uses both to replace Sanad's baseline router and to add three new parts. It also shows the price of rules: they fit the tickets they were written from.

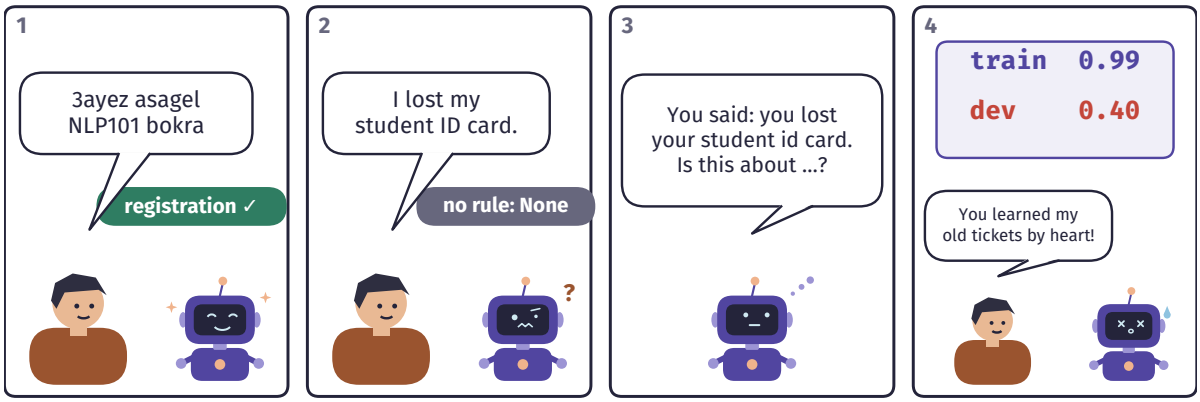


Figure L2.1 Hand-written rules route Nour's Arabizi registration correctly but have no rule for a lost ID card, so the clarifier asks instead. They score 0.99 on the training tickets they were written from and 0.40 on dev. The numbers are the reference run.

L2.1 What you build

- `normalize/v1_policies.py`, a new shared part: marks and tatweel (P1–P2), letter variants (P3–P4), digits (P5), elongation (P6), and case (P7).
- `slots.py` (course code and amount) and `clarify.py` (reflect and reply).
- `router/v1_rules.py` replaces `v0_majority`. Search stays `v0_exact` until level 3.

L2.2 Steps, in four checkpoints

Figure L2.2 shows where you are going: one request passing through every part you build at this level. Work through the checkpoints in order. Each one ends with a check in `tests/level2/test_public_checkpoints.py`, which ships with your starter; do not start the next checkpoint until it passes. The other level-2 test files are graded later and show as skipped, so a passing check means "ready to move on", not "full marks".

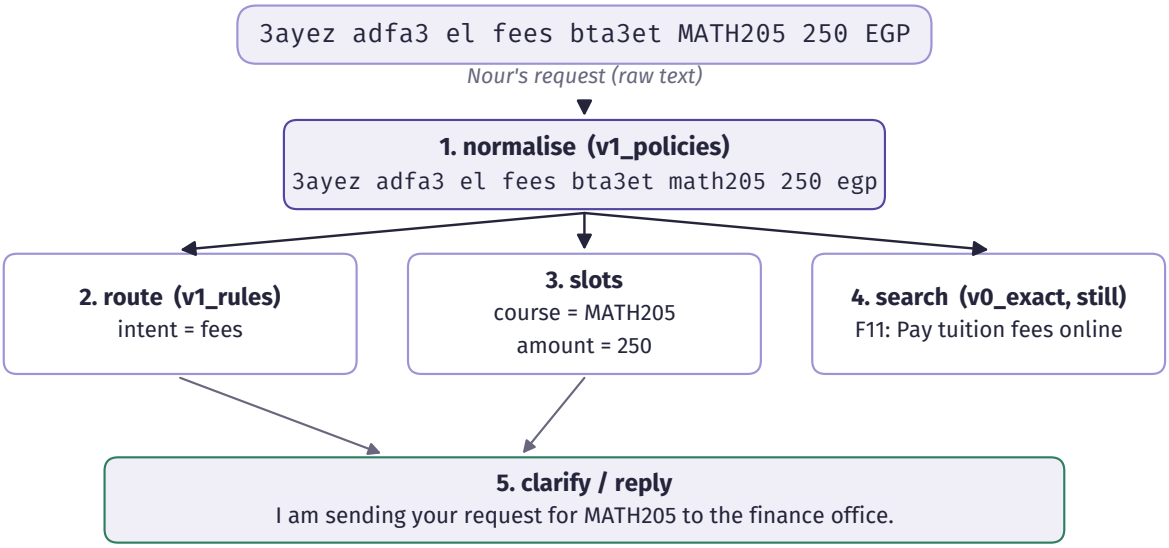


Figure L2.2 One request through Sanad's level-2 parts, from the reference run. The normaliser feeds the router, the slot extractors, and search, which is still exact at this level; the clarifier writes the reply from the intent and the course.

1. **Normalisation.** Implement the normaliser (P1–P7). Predict each output first. Why must P6 run after P3–P4, and what does P6 do to a laugh written *hhhh*?
   `python -m pytest tests/level2/test_public_checkpoints.py -k c1`
2. **Slots and clarification.** Implement `course`, `amount`, `reflect`, and `reply`. The room-number test is marked `xfail(strict=True)`: it records a known failure of the amount policy and must keep failing. Explain in one sentence what the pattern cannot see.
   `python -m pytest tests/level2/test_public_checkpoints.py -k c2`
3. **Routing.** Write the router's `RULES` from the *training* tickets only. The starter already covers *access*; add the other four intents in English, MSA, Egyptian, and Arabizi, for normalised text. Then implement `fit`, `scores`, and `predict`. A ticket goes to the intent with the most matching patterns, a tie goes to the earlier intent, and no match returns `None`. The tests require at least 0.9 accuracy on the training split.
   `python -m pytest tests/level2/test_public_checkpoints.py -k c3`
   Predict the dev accuracy of your rules, then run `python -m sanad report`. Compare training and dev accuracy, count the declined tickets, and read three of them. Are they new words, or new ways of asking?

4. **Comparison report.** Switch `normalize` in `sanad.toml` between `v0_identity` and `v1_policies`, and run `python -m sanad report` each time. Which services change? Then try `python -m sanad route "3ayez asagel NLP101 bokra"` and one message of your own that should make Sanad ask a question. Finish with the full run, `python -m pytest -v`: all three public checks pass, and the graded level-2 tests show as skipped.

L2.3 Your record

Write two entries in `REPLACEMENTS.md`: router `v0→v1` and normaliser `v0→v1`. For each, give the dev scores with the margin, one ticket the new version fixes, one it still fails, and your decision.

Submit: the `pytest -v` output, your `RULES`, the two records, and the two `route` transcripts.

PROJECT LEVEL

L3

Corpus counts and a typo-tolerant search

IN THIS CHAPTER

L3.1 What you build 20

L3.2 Steps, in four checkpoints 20

L3.3 Your record 20

After book Chapter 3: Sections 3.1 and 3.2

Chapter 3 counted what a corpus contains and measured how far apart two spellings are. This level does both for Sanad: it describes the training tickets before you trust any count, and it replaces exact search with a search that forgives a typo, then measures what that tolerance gains and costs.

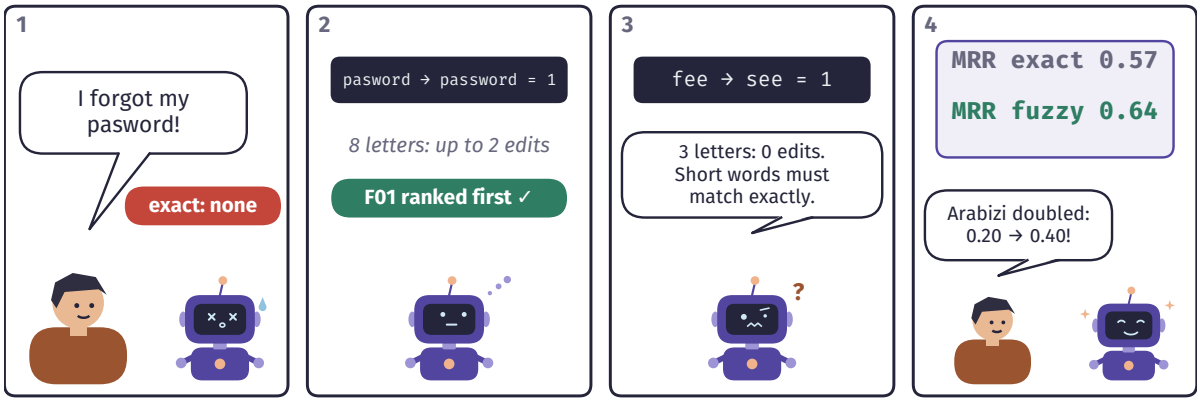


Figure L3.1 Exact search finds nothing for Nour's misspelt *pasword*; fuzzy search allows two edits in an eight-letter word and ranks the password passage first. A three-letter word must still match exactly, because *fee* and *see* are one edit apart. On dev, MRR rises from 0.57 to 0.64, and Arabizi doubles from 0.20 to 0.40. The numbers are the reference run.

L3.1 What you build

- `corpus.py`, a new part: `counts` gives tokens, types, hapax legomena, and the type–token ratio of a list of token lists; `describe` gives those counts per variety and for the whole split. `python -m sanad corpus` prints them.
- `edit.py`: `levenshtein`, with a substitution cost you can change.
- `search/v1_fuzzy.py` replaces `v0_exact`: `max_edits`, `matches`, and `score`.

L3.2 Steps, in four checkpoints

Each checkpoint ends with a check in `tests/level3/test_public_level3.py`, which ships with your starter; do not start the next checkpoint until it passes. The other level-3 test files are graded later and show as skipped.

1. **Counts.** Implement `counts` and `describe`. Every count works on prepared tokens (`pipe.prep`), never on raw text, so it follows the shared normaliser and tokenizer. Predict the counts for book quiz 1 ("pay the fee and the late fee today"), then check.
   `python -m pytest tests/level3/test_public_level3.py -k c1`
   Run `python -m sanad corpus`. Which variety has the highest type–token ratio, and why? Switch `normalize` between `v0_identity` and `v1_policies`: which count changes, and which cannot?
2. **Edit distance.** Implement `levenshtein` in `edit.py`. Check it on the book's `exma`→`exam` and `ktb`→`mktbh`, then with `sub_cost=2`.
   `python -m pytest tests/level3/test_public_level3.py -k c2`
3. **Fuzzy search.** Implement `max_edits`, `matches`, and `score` in `v1_fuzzy.py`. Why must a three-letter word such as *fee* match exactly? Your search must tolerate typos: for the query *pasword*, exact search returns nothing, but fuzzy search should rank F01 (*Reset your portal password*) first.
   `python -m pytest tests/level3/test_public_level3.py -k c3`
4. **Comparison report.** Run `python -m sanad report –part search` and compare `v0_exact` with `v1_fuzzy` by variety. Find one dev ticket that fuzzy search fixes and one that it makes worse, and say why. Finish with `python -m pytest -v`: all three public checks pass, and the graded level-3 tests show as skipped.

L3.3 Your record

Write one entry in `REPLACEMENTS.md`: search `v0`→`v1`, with MRR and its value by variety, one ticket it fixes, one it breaks, and your decision. Add the `python -m sanad corpus` table to the data statement in your README, with the normaliser and tokenizer it used.

Submit: the `pytest -v` output, the corpus table, the search record, and your two example tickets.

PROJECT LEVEL

L4

Units of text, and a first keyboard

IN THIS CHAPTER

L4.1 What you build 23
L4.2 Steps 24
L4.3 Your record 25

After book Chapter 4: Sections 4.1, 4.1.4, 4.1.8, and 4.3

Every Sanad service reads tokens, and so far a token has been whatever lies between two spaces. This level writes that decision down as a policy, learns subword units from the training tickets, and measures how many dev tokens the training vocabulary has never seen. It also starts the third service, the keyboard, with the weakest model that can still be measured.

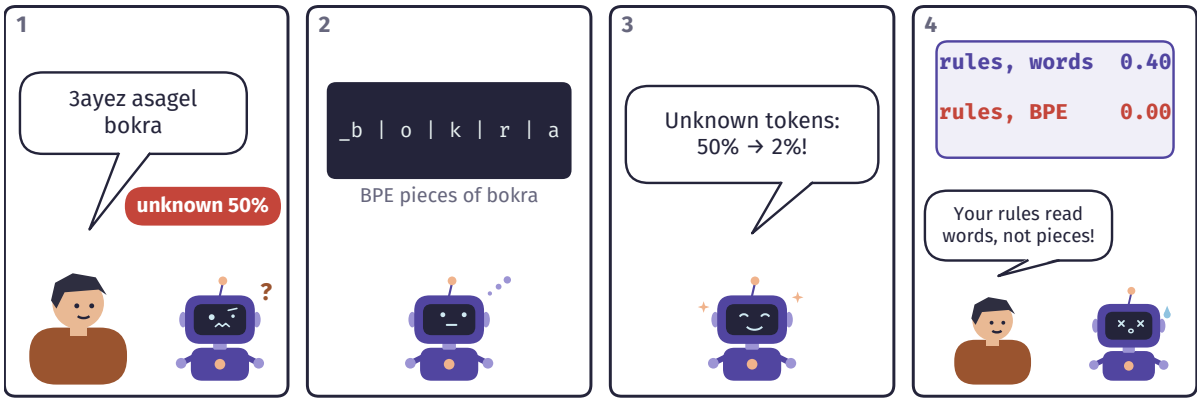


Figure L4.1 A declared tokenizer and BPE change every service at once. BPE leaves almost no unknown tokens, but the rules of level 2 were written for words, so the router stops matching. The numbers are the reference run.

L4.1 What you build

- `normalize/v2_nfc.py`: Unicode NFC, then the level-2 policies.
- `tokenize/v1_policy.py`, a declared tokenizer, and `tokenize/v2_bpe.py`, BPE fitted on the training tickets with a leading word marker.
- `coverage.py`: the unknown-token rate, overall and by variety.

- `keyboard/v0_frequency.py`: suggest the most frequent training tokens. The keyboard's evaluation, `keyboard_report`, is supplied in `evaluate.py`.

L4.2 Steps

1. Unzip `sanad-L4`, set it up, and run the tests once. Record how many level-4 tests fail before you write any code.
2. Implement `v2_nfc.normalize`. The test writes أحمد (*Ahmad*) as a bare alef followed by a combining hamza, U+0654. Before running it, say why normaliser v1 cannot unify that spelling (Section 4.1.4).
3. Implement `Tokenizer.tokenize` in `v1_policy.py` from the docstring's three alternatives. The test `test_policy_known_cost` records that *wi-fi* becomes two tokens. Which service could pay for that decision, and how would you find out?
4. Implement `merge`, `fit`, and `encode_word` in `v2_bpe.py`, following Section 4.1.8. Predict the five merges for the toy corpus of `test_bpe_learns_from_training_text` by hand, then explain why `fit` stops at a pair seen only once.
5. Implement `unknown_rate` and `unknown_by_variety`, then the frequency keyboard. In `sanad.toml`, add the line `keyboard = "v0_frequency"`, then try `python -m sanad suggest "i forgot my"`. Why are the three suggestions the same whatever you type?
6. Replace the shared parts one at a time, and run `python -m sanad report` after each change:
   (a) normaliser `v1_policies` → `v2_nfc`;
   (b) tokenizer `v0_whitespace` → `v1_policy`;
   (c) tokenizer `v1_policy` → `v2_bpe`.
   For each change, list which services changed and in which direction. Find the dev ticket whose search result changes under the declared policy.
7. Read the keyboard rows across tokenizers. The `PPL` column is perplexity, which Chapter 5 defines (Section 5.2.1). For now, read it as the number of equally likely choices the model is, on average, as uncertain as: a model that gives every target probability $1/100$ has perplexity 100, and lower is better. Under BPE the perplexity falls sharply. Count the targets in each row and say what one target is under each tokenizer, then explain why the two numbers cannot show which keyboard is better. Level 5 returns to this comparison, after Chapter 5 has defined perplexity.

L4.3 Your record

Write three entries in `REPLACEMENTS.md`: normaliser `v1→v2`, tokenizer `v0→v1`, and tokenizer `v0→v2`. The keyboard is new, so record its scores as the baseline later keyboards must beat, together with the unknown-token rate by variety. Then decide which tokenizer every service will share. A tokenizer that helps one service and harms another is a decision, not a measurement: say which service you favour and why.

Submit: the `pytest -v` output, the three records, your toy-BPE prediction, the unknown-rate table, and one paragraph on the shared-tokenizer decision.

APPENDIX

A

Project data and configuration

IN THIS CHAPTER

Tickets

Each line of `data/tickets_{train,dev,test}.jsonl` is one ticket: `id`, `text`, `intent` (access, assessment, registration, fees, other), `variety` (en, msa, egy, arabizi), `course` and `amount` (or `null`), and `faq`, the ids of the passages that answer it. The split is fixed: within each variety and intent, tickets are dealt train, train, train, dev, test. Every split therefore covers every variety and intent; the intents are balanced (15, 4, and 4 of each).

Split	English	MSA	Egyptian	Arabizi	Total
train	25	20	15	15	75
dev	5	5	5	5	20
test	5	5	5	5	20

FAQ passages

`data/faq.jsonl` holds 18 procedures, F01–F18, each with an English title, English text, and Arabic text. Search judges a passage relevant when its id is in the ticket's `faq` list.

`sanad.toml`

```
[parts]
normalize = "v2_nfc"       # sanad/normalize/v2_nfc.py
tokenize  = "v1_policy"    # sanad/tokenize/v1_policy.py
router    = "v2_logreg"
search    = "v1_fuzzy"
keyboard  = "v1_bigram"
```

This is the reference configuration at release, after level 7: every line is a version the dev evidence kept, not simply the newest one. A checkpoint's `sanad.toml` selects the reference decisions of the levels before it, so it only names parts that already work. `python -m sanad report` scores every version present, whatever the file selects. The settings chosen on dev, `THRESHOLD` in `router/v2_logreg.py` and `LAMBDA` in each keyboard, live in their modules.

Replacement record

```
## Level _ : <part> <old version> -> <new version>
- Split and n:
- Shared parts (normalize, tokenize):
- Scores, old -> new (with the 95% margin where it applies):
- One failure the new version fixes (ticket id and text):
- One failure the new version introduces or keeps:
- Decision (keep / replace / revert) and why:
```